HDLBits Attempt 2

Day 4 Questions: Vector, DFF, and PS/2

A single discussion note for the three question-bearing problems in the current Day 4 batch: DFFs and gates, vector part-select, and the PS/2 packet parser with datapath.

8	3	1
newly completed problems	question groups answered	combined review PDF

Core mental model

Part-select syntax chooses bits, a D flip-flop stores the next state presented at D, and a clocked FSM datapath must have one procedural owner for each register.

- Problem 67: use indexed part-selects with byte-sized strides and valid starting indices.
- Problem 66: the D-input excitation is simply the desired next Q value.
- Problem 70: separate next-state logic from registered state/data updates.
- Tracker rows for these three problems share the same yellow question-note color and link to this PDF.

Indexed part-selects and byte reversal

The target is to reverse four 8-bit bytes in a 32-bit vector without reversing the bit order inside each byte.

The one-line rule

Use an 8-bit stride because each slice is a byte. The destination grows upward with **+:**; the source starts at the high bit and grows downward with **-:**.

How the syntax expands

Form	Meaning	Example
base +: width	Start at **base** and select upward.	out[i*8 +: 8]
base -: width	Start at **base** and select downward.	in[31-i*8 -: 8]

Loop trace

i	Destination	Source	Byte moved
0	out[7:0]	in[31:24]	highest input byte -> lowest output byte
1	out[15:8]	in[23:16]	second input byte -> second output byte
2	out[23:16]	in[15:8]	third input byte -> third output byte
3	out[31:24]	in[7:0]	lowest input byte -> highest output byte

Correct Verilog

```
module top_module(
    input  [31:0] in,
    output reg [31:0] out
);
    integer i;
    always @(*) begin
        for (i = 0; i < 4; i = i + 1) begin
            out[i*8 +: 8] = in[31 - i*8 -: 8];
        end
    end
endmodule
```

Example: **in = 32'hAABBCCDD** gives **out = 32'hDDCCBBAA**.

Fast debugging cues

Vector mistakes to catch immediately

- Using **4*i** moves by nibbles, not bytes. A byte step is **8*i**.
- A start such as **4*i-1** becomes -1 when i=0, creating an invalid select.
- Write **-:** as one operator. A space between '-' and ':' changes the parse.
- An output assigned inside **always @(*)** must be declared **reg** in Verilog.
- Assign every output bit on every combinational path so no latch is inferred.

> **Width check**
>
> Both sides of the assignment select exactly 8 bits. That single width check catches most indexed part-select mistakes before simulation.

D flip-flop excitation

For a D flip-flop, the value present on D at the active clock edge becomes Q after the edge. Therefore the required input for any transition is the desired next state.

Characteristic equation	**Q(n+1) = D**
Excitation equation	**D = Q(n+1)**

Present Q(n)	Desired Q(n+1)	Required D	Interpretation
0	0	0	hold 0
0	1	1	set
1	0	0	reset
1	1	1	hold 1

> **Memory cue**
>
> A D flip-flop has no separate set/hold/reset excitation coding. Put the state you want next on D; the current Q does not change that rule.

Separate state decisions from stored data

The parser recognizes a three-byte packet whose first byte has in[3] = 1, stores the bytes, and raises done for the completed packet.

Primary bug

The register **data** was assigned from more than one procedural block. A synthesizable register should have one procedural owner; here, the clocked block owns both **state** and **data**.

Clean two-process structure

Block	Responsibility	Assignments
Combinational always @(*)	Decide the next state from the current state and input.	next_state only
Sequential always @(posedge clk)	Commit state and store bytes at clock edges.	state and data
Continuous assign	Expose Moore-style outputs from registered state/data.	done and out_bytes

State transitions

Current state	Condition	Next state	Data action at this edge
idle	in[3] = 1	sbyte2	store byte 1 in data[23:16]
idle	in[3] = 0	idle	hold data
sbyte2	any input	sbyte3	store byte 2 in data[15:8]
sbyte3	any input	donee	store byte 3 in data[7:0]
donee	in[3] = 1	sbyte2	start next packet; replace byte 1
donee	in[3] = 0	idle	hold completed packet

Why byte 1 is stored in idle

At the edge where the first valid byte is sampled, the current state is still **idle**. The nonblocking assignment to state does not take effect until after the edge. Therefore the byte-1 write condition must test the old state: **state == idle && in[3]**.

Problem 70

Correct parser and datapath implementation

```
module top_module(
    input clk,
    input [7:0] in,
    input reset,
    output [23:0] out_bytes,
    output done
);
    reg [23:0] data;
    parameter idle   = 2'b00;
    parameter sbyte2 = 2'b01;
    parameter sbyte3 = 2'b10;
    parameter donee  = 2'b11;
    reg [1:0] state, next_state;

    always @(posedge clk) begin
        if (reset) begin
            state <= idle;
            data <= 24'd0;
        end else begin
            state <= next_state;
            if (state == idle && in[3])
                data[23:16] <= in;
            else if (state == sbyte2)
                data[15:8] <= in;
            else if (state == sbyte3)
                data[7:0] <= in;
            else if (state == donee && in[3])
                data[23:16] <= in;
        end
    end

    always @(*) begin
        next_state = state;
        case (state)
            idle: begin
                if (in[3]) next_state = sbyte2;
                else       next_state = idle;
            end
            sbyte2: next_state = sbyte3;
            sbyte3: next_state = donee;
            donee: begin
                if (in[3]) next_state = sbyte2;
                else       next_state = idle;
            end
            default: next_state = idle;
        endcase
    end

    assign done = (state == donee);
    assign out_bytes = data;
endmodule
```

Hold behavior is implicit

There is deliberately no final **else data <= 0;**. When no byte-write condition is true in a clocked block, the register retains its previous value.

Timing trace and final checklist

Three-byte packet example

Assume the input bytes arrive on consecutive active clock edges as A8, 34, and F2.

Edge	State before edge	Input	Write	State after edge	data after edge	done
1	idle	A8	data[23:16]	sbyte2	A8 00 00	0
2	sbyte2	34	data[15:8]	sbyte3	A8 34 00	0
3	sbyte3	F2	data[7:0]	donee	A8 34 F2	1
4	donee	00	none	idle	A8 34 F2	0

Back-to-back packet restart

If the input in **donee** already has bit 3 set, that byte is the first byte of the next packet. The FSM goes directly to **sbyte2** and replaces data[23:16] at the same edge.

What caused the earlier failure

- Assigning **data** in both combinational and sequential logic created multiple procedural drivers.
- Waiting until **sbyte2** to store byte 1 was one edge late; the current state at the first byte is idle.
- Clearing data in an unmatched clocked **else** erased a partially or fully assembled packet.
- Keeping **donee** as the next state when in[3] = 0 left done asserted indefinitely.

Final submission checklist

1	One procedural owner for every register
2	next_state gets a default before the case statement
3	Byte writes test the current state at the sampling edge
4	done is derived only from the registered donee state